

Assignment 2, Combinatorial Optimization, Due by 12 October 2026 (23.59). Hand in through canvas.

- To be made in groups of at most 3.
- Read the documents *Plagiarism* and *AI in this course* in the course manual module on Canvas.
- Put your answers in a pdf file and include the code for Questions 4,5, and 6 in separate files. Text should be typed (latex, word, ...). Illustrations (if any) can be drawn by hand if you wish.
- Your pdf document should:
 - have your names and group number on the front page.
 - use at most 1 page for each question (A4 format, font size 10, normal margins, and a readable layout).
 - start each question on a new page.
 - have (as an appendix) a note on the work division, the use of AI, and any other references used (if any).

Points may be subtracted if your document does not fulfill these rules.

- This assignment counts for 30% of the final grade.
- The number of points indicated gives the maximum number for each part. Your grade is $\text{YourTotal}/5$.

Question 1: (10 points)

Consider again the optimization problem of question 1 of Assignment 1, which we shall call the Santa Claus problem.

SANTA CLAUS:

- Instance:* Integers m (machines) and n (jobs) and integer job lengths $p_{ij} \geq 0$, which is the length of job j if scheduled on machine i , for all $i \in \{1, 2, \dots, m\}$ and $j \in \{1, 2, \dots, n\}$
- Solution:* Assignment of jobs to machines: $x_{ij} = 1$ if job j is assigned to machine i and $x_{ij} = 0$ otherwise.
- Value:* Minimum machine load: $\min_i \sum_{j=1}^n p_{ij} x_{ij}$
- Goal:* Maximize value

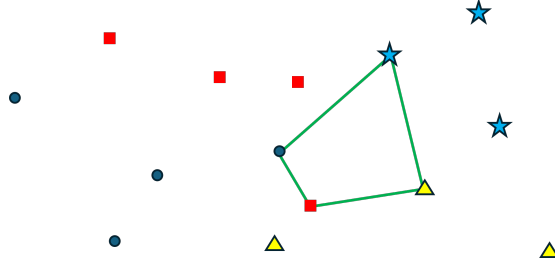
$$\begin{aligned} \text{(ILP) max } & Z \\ \text{s.t. } & Z \leq \sum_{j=1}^n x_{ij} p_{ij} \quad \text{for all } i \in \{1, 2, \dots, m\} \\ & \sum_i x_{ij} = 1 \quad \text{for all } j \in \{1, 2, \dots, n\} \\ & x_{ij} \in \{0, 1\} \quad \text{for all } i \in \{1, 2, \dots, m\} \text{ and } j \in \{1, 2, \dots, n\}. \end{aligned}$$

Assume that there are only two persons to share the items ($m = 2$). Moreover, for any item, the valuation of both persons is exactly the same ($p_j := p_{1j} = p_{2j}$ for all j).

- (a) Give a polynomial time algorithm that has a constant factor approximation guarantee. (Just describe the algorithm, the proof is for the next sub-question.)
- (b) Prove that your algorithm is indeed a constant factor approximation algorithm.

Question 2: (5 points)

Consider the Group-TSP problem which was discussed in the guest lecture by Dr. van Ee. In Group-TSP, one is given a vertex set V with distance (or weight) $w(e) \geq 0$ for all $e \in E$. We assume the graph is complete and that the distance function w satisfies the triangle inequality. Further, we are given a set of groups $V_1, V_2, \dots, V_S \subseteq V$ of points. The goal is to find a tour of minimum length (weight) which contains at least one point from each group.



Clearly, if each group contains exactly one point we have the standard TSP, so the problem is NP-hard in general. In the lecture, an $O(k)$ -approximation was presented, where k is the (maximum) number of points in each group. The algorithm is based on LP-rounding. However, one step in the proof is missing from the slides. It is mentioned that "Using the triangle inequality, we can adjust $\bar{x}, \bar{y}$ such that it is a feasible solution to the following LP for TSP."

In this question you are asked to complete the proof by showing how this can be done. However, we simplify the question by omitting the degree constraint. So, let $(\bar{x}, \bar{y})$ be the rounded solution for the LP-relaxation for group-TSP. Show how to obtain a feasible solution x to the following LP for TSP:

$$\begin{aligned} \text{(ILP) min} \quad & \sum_{e \in \bar{E}} x_e w(e) \\ \text{s.t.} \quad & \sum_{e \in \delta(U)} x_e \geq 2 \quad \text{for all } U \subset \bar{V} \text{ with } U \neq \emptyset, U \neq \bar{V} \\ & x_e \geq 0 \quad \text{for all } e \in \bar{E}. \end{aligned}$$

N.B. Note that we use $\bar{V} \subseteq V$ and $\bar{E} \subseteq E$ in stead of V and E . In your answer, also explain what is meant by $\bar{V}$ and $\bar{E}$.

(Side remark: Note that the only difference with the LP on the slides is that the degree constraint ('=2') is missing here. It can be shown that any solution to the LP above can be converted into a solution for the LP with the degree constraints included without an increase in cost. However, this final step you do not have to show here.)

Question 3: (10 points)

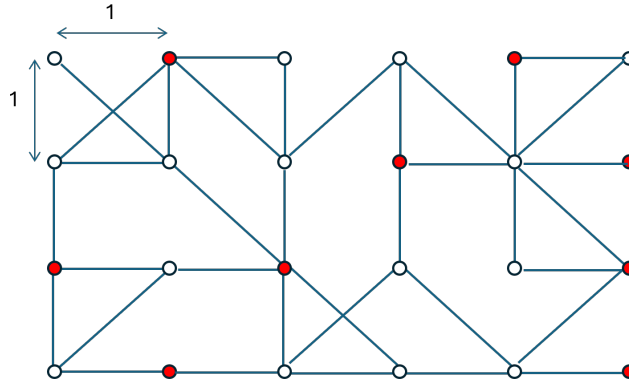


Figure 1: An example of a 4×6 grid graph with a (non-optimal) independent set in red.

MAXIMUM INDEPENDENT SET FOR GRID GRAPHS:

- Instance:* An m and n grid graph $G = (V, E)$ where there is a vertex for each of the mn grid points and any edge has the property that it is between two points at distance either 1 or $\sqrt{2}$.
- Solution:* A subset $S \subseteq V$ such that no edge has both endpoints in S .
- Value:* The number of points in S .
- Goal:* Maximize value.

- (a) Describe a PTAS for this problem. (Just describe. The proof is for (b).)
- (b) Prove that your algorithm is indeed a PTAS. That means: show that for every $\epsilon > 0$ and every instance of the problem, the algorithm returns a solution that contains at least $1 - \epsilon$ times the value of the largest independent set. Argue that the algorithm's running time is polynomial in m and n .

Hint 1: For your running time analysis, use that ϵ is a constant.

Hint 2: Use that the problem can be solved exactly and in constant time if the number of vertices is a constant. For example, if there are only $f(\epsilon)$ vertices for some function f then a maximum independent set can be found efficiently.

Question 4: (5 points)

We have seen a greedy algorithm for max 2-cut (Chapter 5). The algorithm can easily be generalized to max k -cut: assign vertices one-by-one to one of the k sets and always choose a set that maximizes the number of edges added to the cut.

The code in the file `kcut.py` defines a random graph on $n = 1000$ vertices implicitly by the `edge` function. Replace the seed number 42 by your group number.

Run the greedy k -cut algorithm for $k = 2, 3, 4, \dots$. Let ALG_k be the number of edges in the k -cut returned by the algorithm. Continue until for some $k = k'$ all edges are in the cut. In your report, plot k against ALG_k for $k = 2, 3, \dots, k'$.

Question 5: (10 points)

In the k -cluster problem we need to partition a given point set into k sets (clusters). The diameter of a cluster is the maximum distance between 2 points in the cluster and the objective is to minimize the maximum diameter among the clusters. We have seen in the exercises of week 2 that the greedy k -center algorithm gives a 2-approximation for the k -cluster problem

There is a simple local search algorithm for k -clustering which works as follows: Given a clustering, it first computes for each cluster the center point (which does not have to be an input point). The center point of a cluster S is the point x that minimize $\max_{v \in S} d(x, v)$. Next, each input point is assigned to its nearest center point, which then defines the new clustering. This is repeated until there is no change in the solution anymore. This is known as Lloyd's algorithm. (N.B. There are several variants of Lloyd.)

The code in the file `kcluster.py` selects 20 times a random list of 1000 points in the plane.

Run an experiment where you apply Lloyd's local search algorithm 3 times:

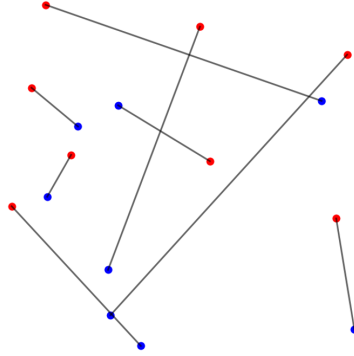
- once with the greedy k -center solution as starting solution,
- once with the first k points from the list as starting solution,
- and once with k random points from the list as starting solution.

Use $k = 50$ and $n = 1000$. Repeat this 20 times and compare the results.

N.B. Use the point set as generated by the file but with the seed value 42 replaced by your group number.

Question 6: (10 points)

In the Euclidean, bipartite mincost matching problem we are given n red points and n blue points in the Euclidean plane. Goal is to find a red-blue perfect matching of smallest total distance. Matching in general can be solved in polynomial time, however, when the number of points n is large then even an $O(n^2)$ algorithm is useless. Consider the following 3 approaches:



- 1 **Optimize:** Compute an optimal solution.
- 2 **Greedy:** Go through the red vertices in arbitrary order, each time match the red vertex to its nearest unmatched blue vertex.
- 3 **Random:** Take a random matching.

Clearly, algorithm 1 performs best and algorithm 3 performs the worst. But for the running time 3 is best and 1 is worst. For large n , say $n = 10^6$, an optimal solution is very hard to find but the greedy algorithm might still work (depending on the implementation), and the random matching is found very quickly.

Come up with your own algorithm which you expect to be both fast and efficient (like greedy or better) and describe the main idea of your algorithm. Any algorithm is allowed and you do not have to prove anything on its performance. Just describe the main idea.

Next, implement the 3 algorithms plus your own algorithm and test their performance (time and value) on random instances of n points for $n = 50, 100, 150, 200$. Put your results in a table. For example like the table below.

		$n = 50$	$n = 100$	$n = 150$	$n = 200$
Optimize	average				
	best				
Greedy	average				
	best				
Random	average				
	best				
Own	average				
	best				